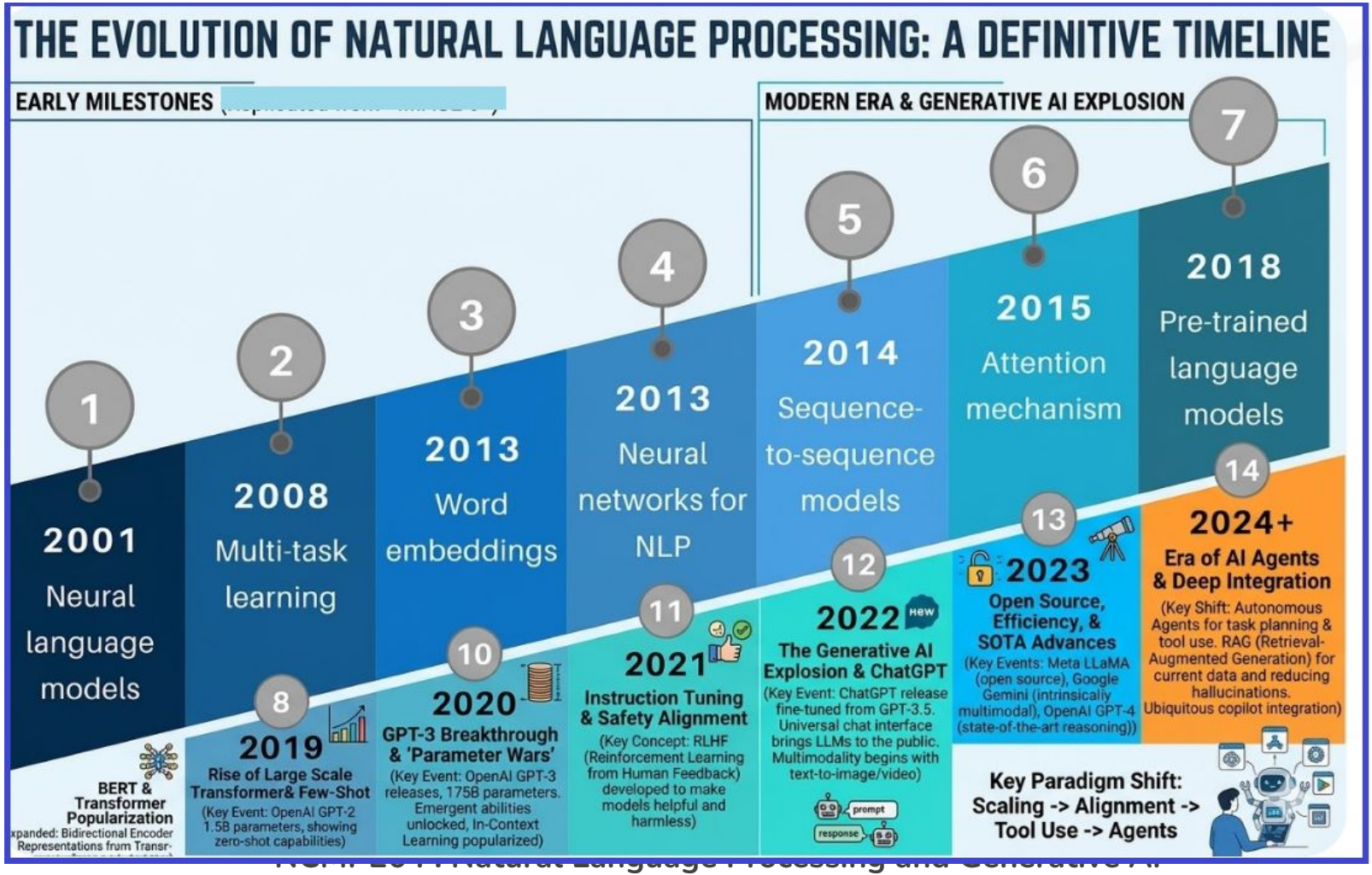
Module 5 : Transformers & LLMs

Dr. Priya R L, Mrs. Sunita Suralkar
Faculty Incharge for NCMPE64 (Natural Language Processing & Generative AI)
Department of Computer Engineering
VES Institute of Technology, Mumbai

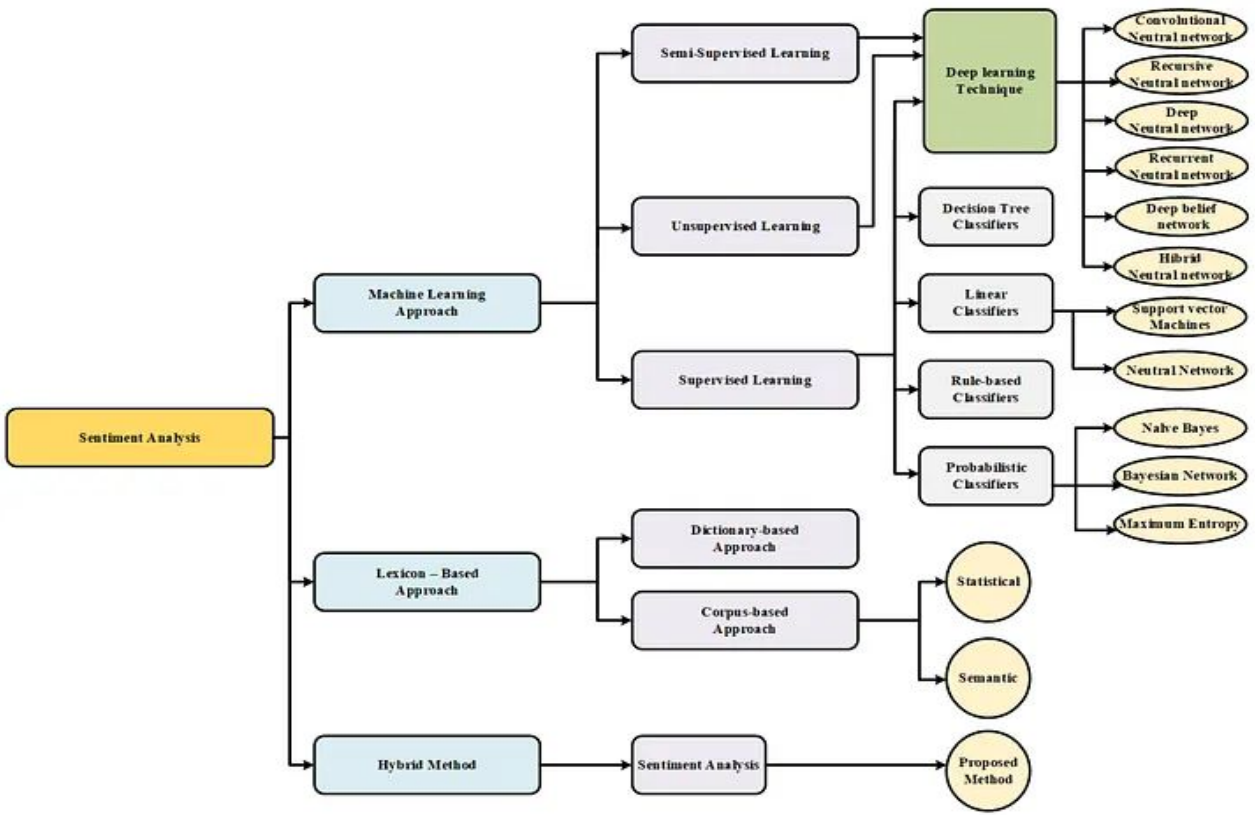
Agenda - Part : I

- Transformer Architecture and
- Attention Mechanism,
- Popular LLMs: BERT,
- GPT-4,
- RoBERTa,
- Meta LLaMA2,
- Google PaLM2

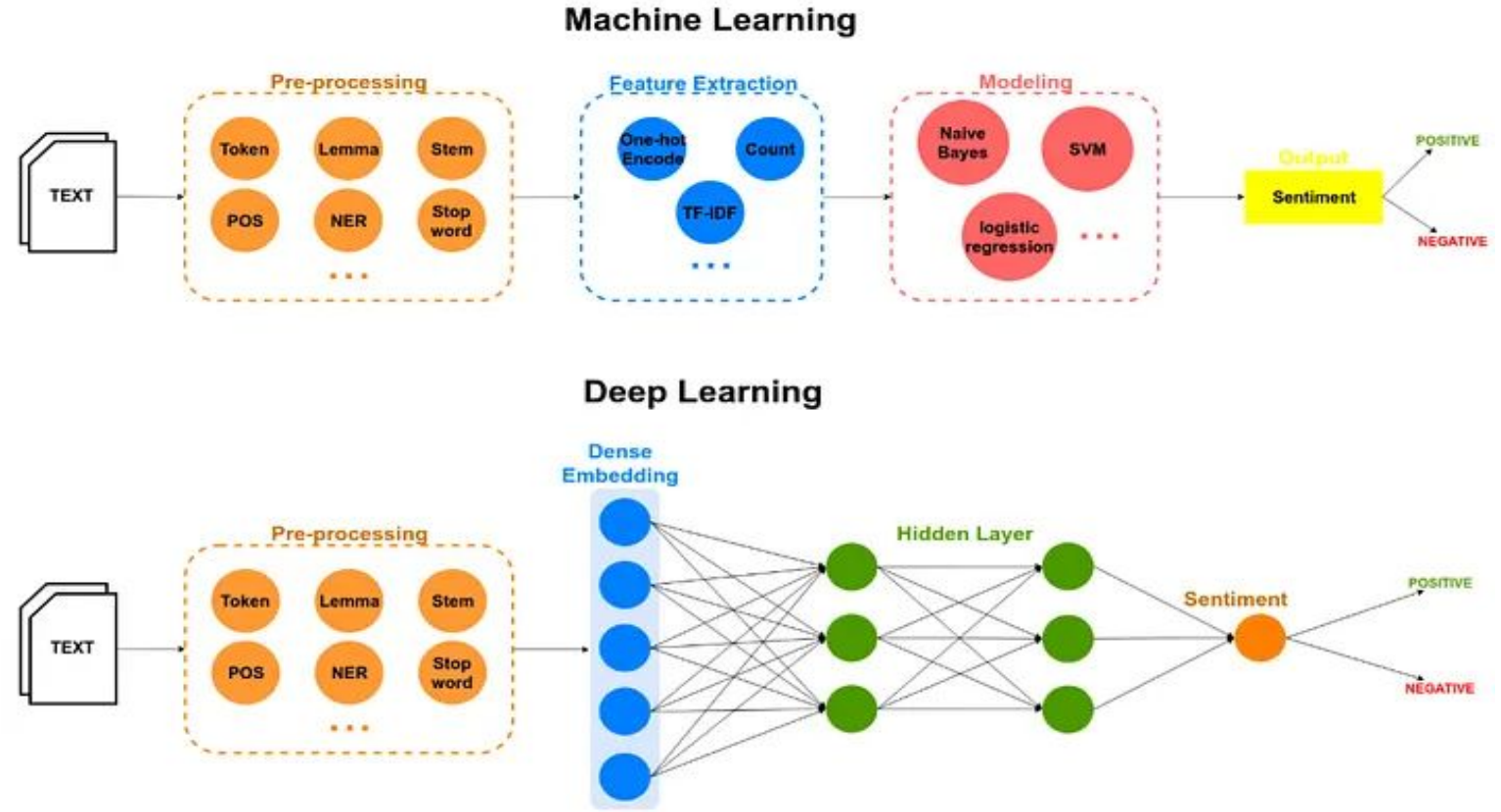

THE EVOLUTION OF NATURAL LANGUAGE PROCESSING: A DEFINITIVE TIMELINE
EARLY MILESTONES
MODERN ERA & GENERATIVE AI EXPLOSION
1
2001
Neural language models
2
2008
Multi-task learning
3
2013
Word embeddings
4
2013
Neural networks for NLP
5
2014
Sequence-to-sequence models
6
2015
Attention mechanism
7
2018
Pre-trained language models
8
2019
Rise of Large Scale Transformer& Few-Shot
(Key Event: OpenAI GPT-2 1.5B parameters, showing zero-shot capabilities)
BERT & Transformer Popularization
10
2020
GPT-3 Breakthrough & 'Parameter Wars'
(Key Event: OpenAI GPT-3 releases, 175B parameters. Emergent abilities unlocked, In-Context Learning popularized)
11
2021
Instruction Tuning & Safety Alignment
(Key Concept: RLHF (Reinforcement Learning from Human Feedback) developed to make models helpful and harmless)
12
2022
NEW
The Generative AI Explosion & ChatGPT
(Key Event: ChatGPT release fine-tuned from GPT-3.5. Universal chat interface brings LLMs to the public. Multimodality begins with text-to-image/video)
prompt
response
13
2023
Open Source, Efficiency, & SOTA Advances
(Key Events: Meta LLaMA (open source), Google Gemini (intrinsically multimodal), OpenAI GPT-4 (state-of-the-art reasoning))
14
2024+
Era of AI Agents & Deep Integration
(Key Shift: Autonomous Agents for task planning & tool use. RAG (Retrieval-Augmented Generation) for current data and reducing hallucinations. Ubiquitous copilot integration)
Key Paradigm Shift: Scaling -> Alignment -> Tool Use -> Agents

	Feature	Statistical(N-gram) Language Models	Neural Language Models
1	**Representation**	Uses **discrete representations**. Words are treated as unique IDs; the model doesn't know "cat" and "dog" are both animals.	Uses **dense vectors (embeddings)**. Words are mapped to a continuous space where similar words have similar values.
2	**Context Window**	**Fixed and limited**. Usually only looks at the previous 2–5 words (N-1). It cannot remember the beginning of a long sentence.	**Flexible and long-range**. Architectures like Transformers can process thousands of words simultaneously to maintain context.
3	**Data Sparsity**	**High**. If a specific sequence of words wasn't in the training data, the model gives it a 0% probability (requires "smoothing" to fix).	**Low**. Because it uses embeddings, it can predict likely words even if it hasn't seen that exact sequence before by using "neighboring" concepts.
4	**Generalization**	**Poor**. It struggles with synonyms or paraphrasing because it looks for exact word matches.	**Excellent**. It generalizes well across different writing styles and understands that "He is happy" is similar to "He is joyful."
5	**Computational Cost**	**Low**. Very fast to train and run; can often be handled by a standard laptop in seconds.	**High**. Requires massive computational power (GPUs/TPUs) and significant time to train.
6	**Model Size**	**Scales poorly**. As the context window (N) increases, the size of the probability table grows exponentially, making it impractical.	**Scales efficiently**. The number of parameters grows, but the model remains a compact mathematical function compared to a massive raw table.

Case Study: Sentiment Analysis

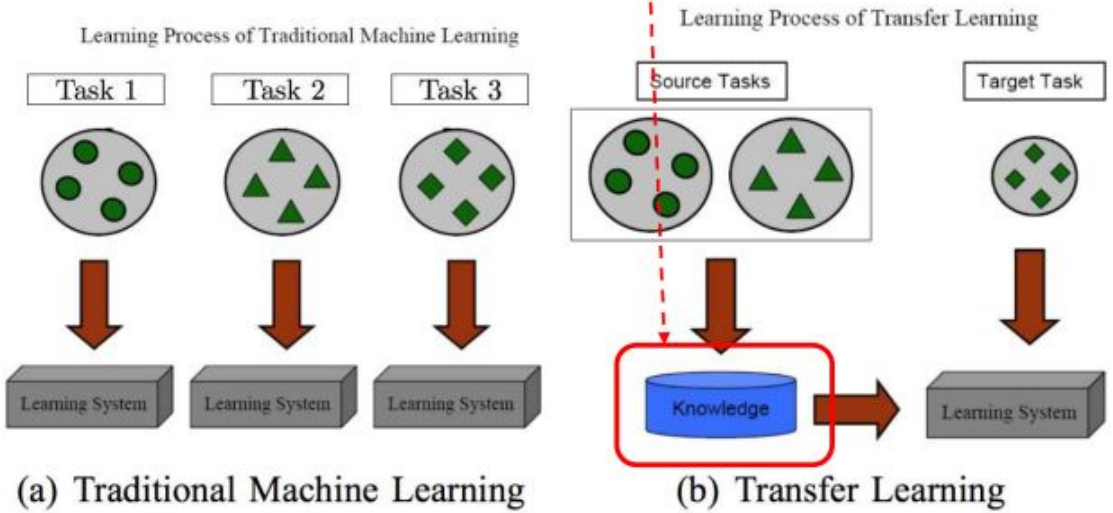


Case Study: Sentiment Analysis (ML vs DL)



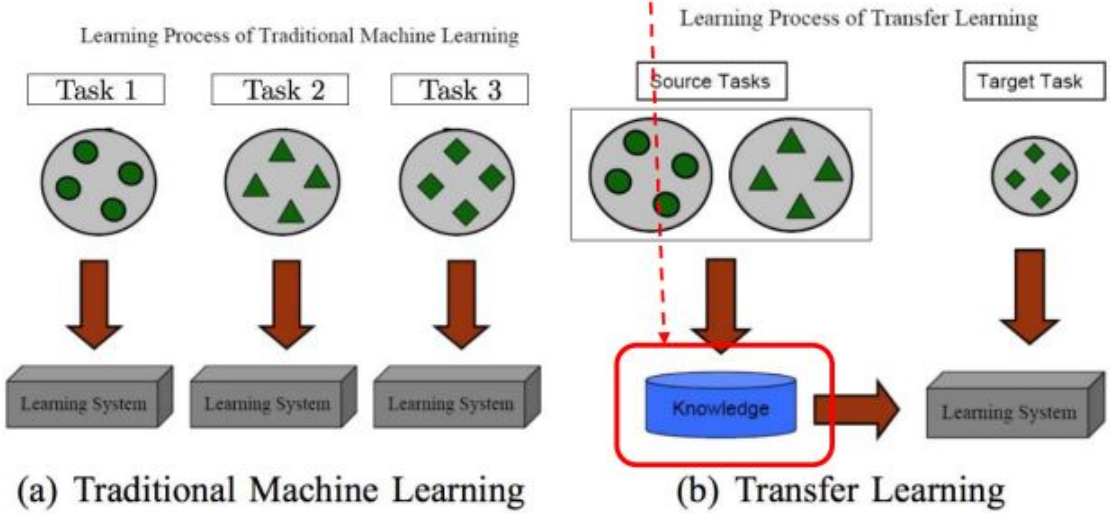
Pre-trained Language Representations

- **How can we take advantage of distributed word representation?**
  - Transfer Learning
- **What is Transfer Learning?**



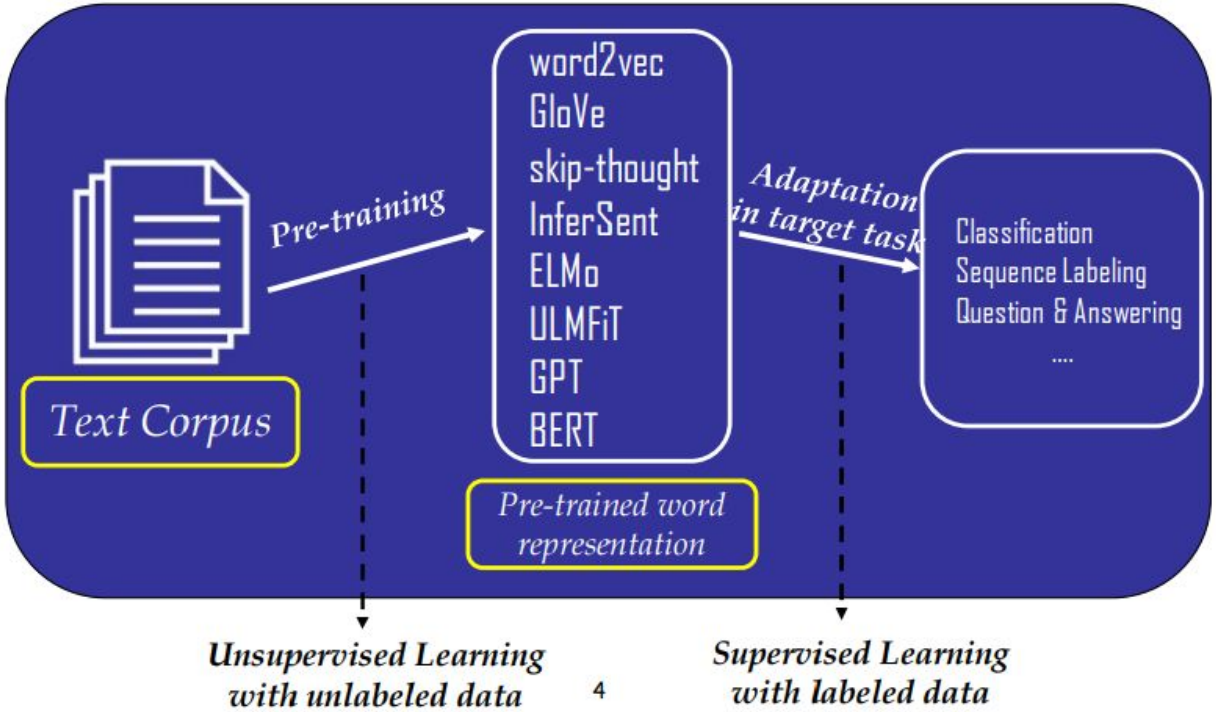
Pre-trained Language Representations

- **How can we take advantage of distributed word representation?**
  - Transfer Learning
- **What is Transfer Learning?**



Pre-trained Language Representations

- **Transfer Learning using word representations**
  - Pre-trained Word Representation

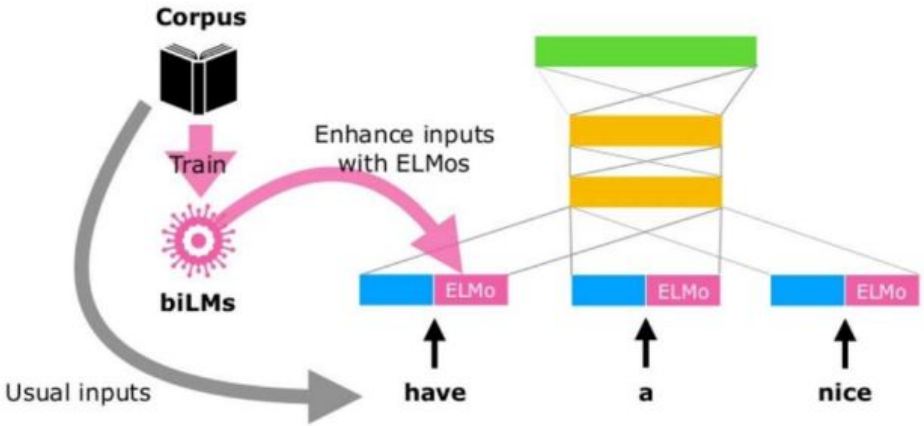


Pre-trained Language Representations

- **Two kinds of Pre-trained Language Representation:**
  - Feature-based Approach
  - Fine-tuning Approach

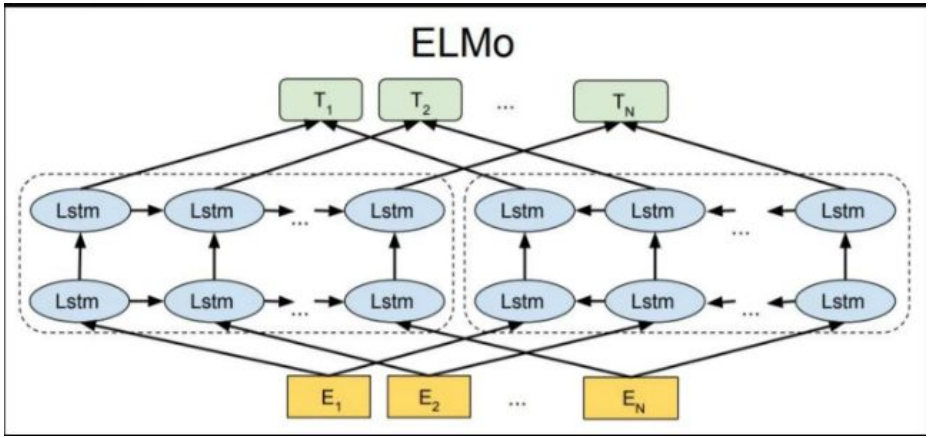
❖ **Feature-based approach**

- Use task-specific architectures that include the pre-trained representations as additional features
  - Learned representations are used as features in a downstream model
- ELMo



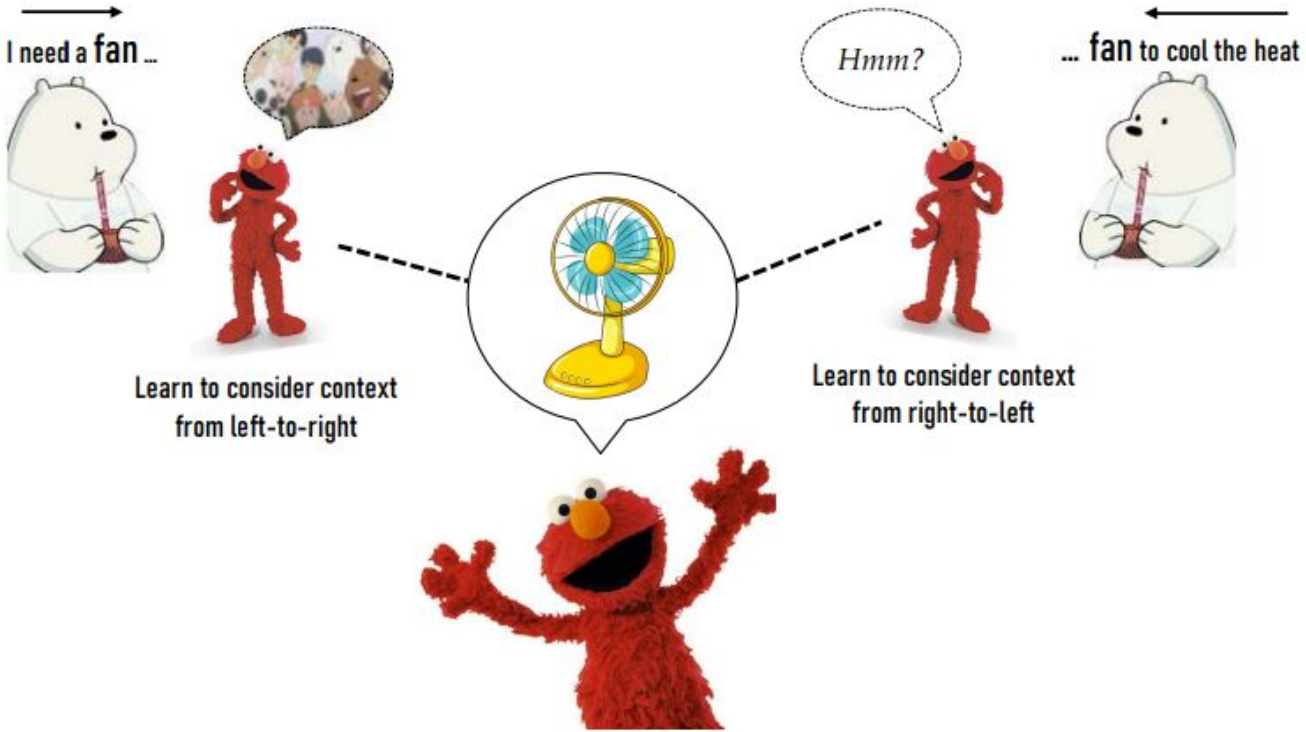
ELMo (Embeddings from Language Models)[2018]

- ELMo is a **deep contextualized word representation** that models both complex characteristics of word use (e.g., syntax and semantics) and how these uses vary across linguistic contexts (e.g., to model polysemy).
- ELMo generates **context-sensitive embeddings** using **deep bi-directional LSTM** models. It provides different embeddings for words based on their context. Contextual LM - Two-Way, Two-Layered



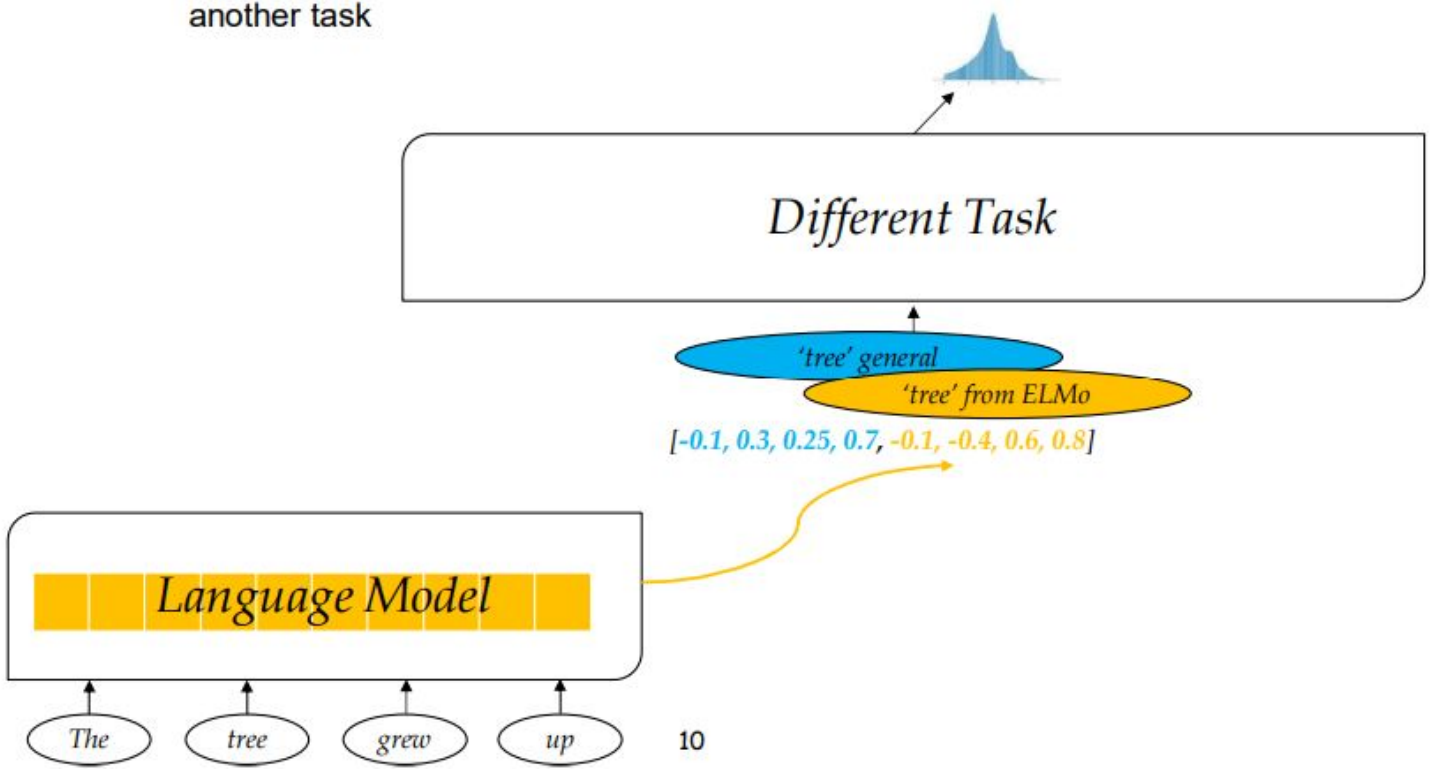
ELMo - Feature Based Approach

- Let the words be represented according to the context !!!

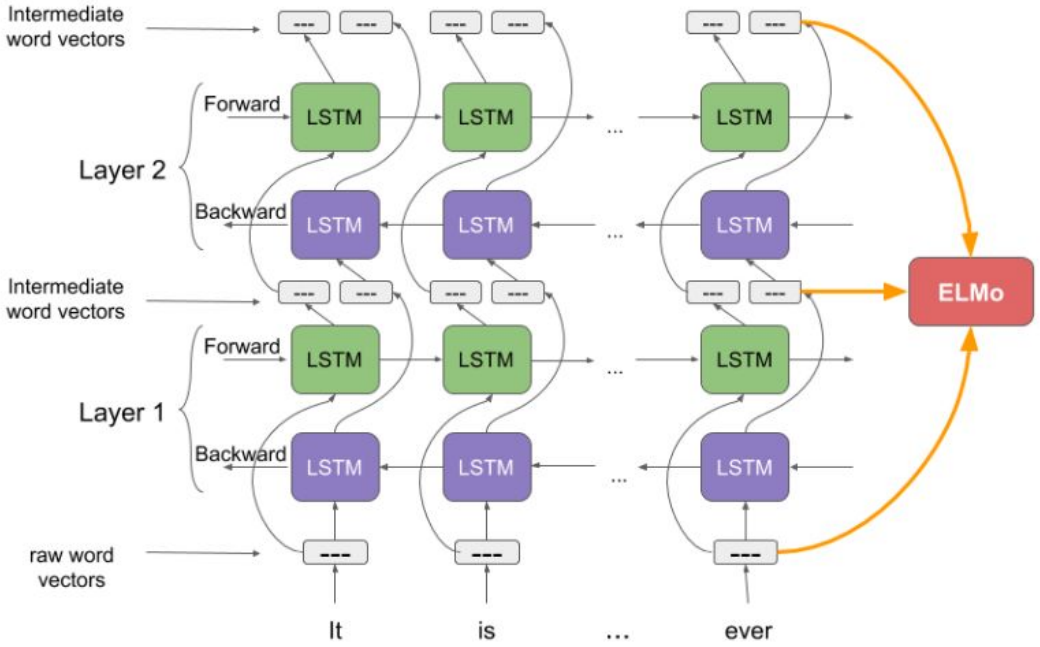


ELMo (Peters et al. 2018) (Cont'd)

- Trained task-specifically
  - Learned parameters(weights) from language model are used as a **feature** for another task



ELMo Inferences



- ELMo word vectors are computed on top of a two-layer bidirectional language model (biLM). This biLM model has two layers stacked together. Each layer has 2 passes — forward pass and backward pass
- Uses a character-level convolutional neural network (CNN) to represent words of a text string into raw word vectors
- As the input to the biLM is computed from characters rather than words, it captures the inner structure of the word.

Key Features : ELMo

- **Contextualized Representations:** ELMo embeddings vary depending on the context in which a word appears. Different from traditional embeddings like Word2Vec or GloVe, which provide a single representation for each word regardless of context.
- **Deep Representations:** ELMo uses deep bidirectional LSTM networks to generate embeddings, capturing complex syntactic and semantic information.
- **Layer-wise Representations:** ELMo combines representations from all the layers of the biLSTM, which allows it to capture different types of information at different layers.
- **Pros:** Contextualized Embeddings, Bi-directional Context. **Cons:** Complexity and Size, Fixed context window.

Bidirectional Encoder Representations from Transformers - BERT

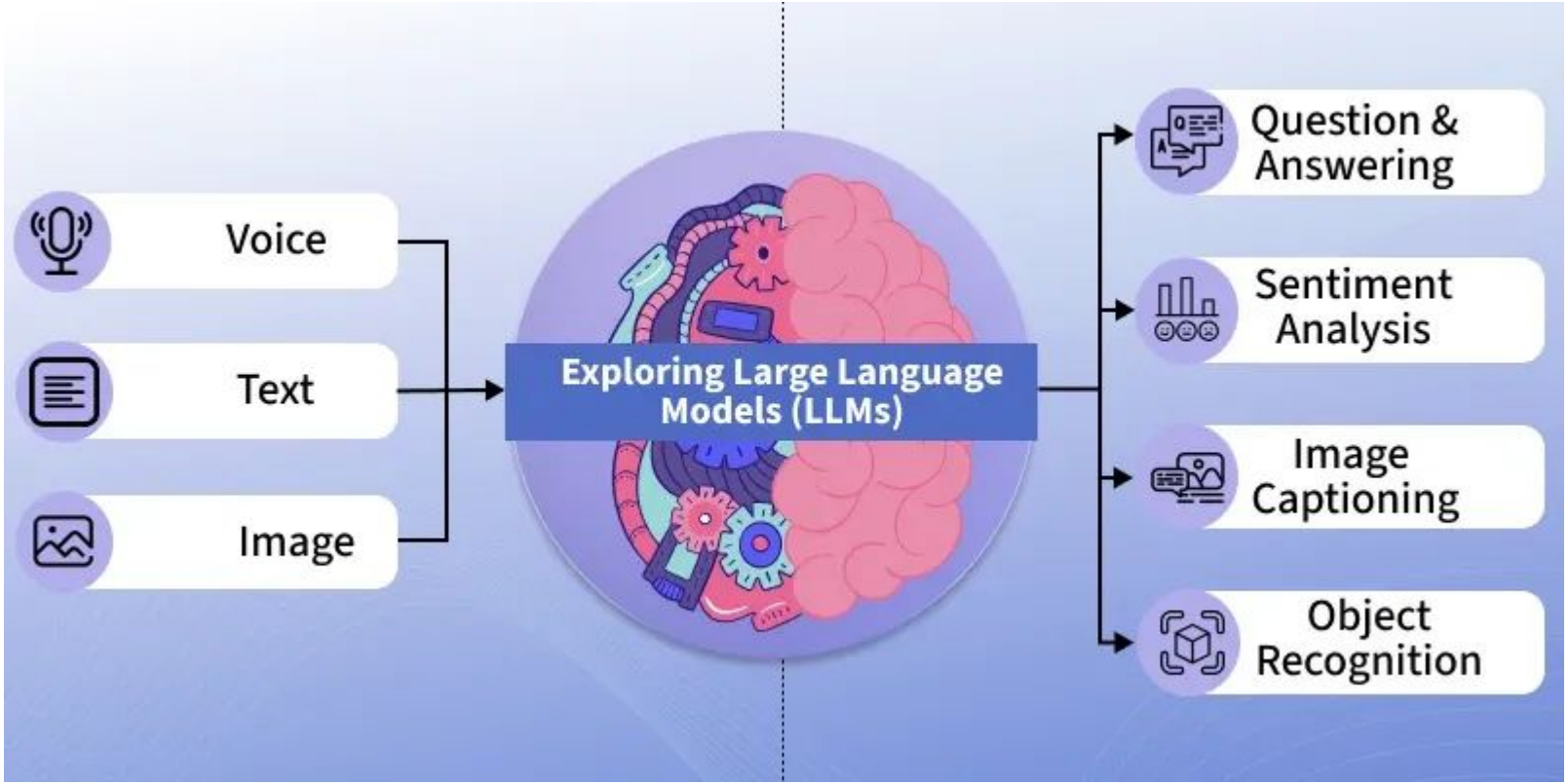
- BERT is a transformer-based model that generates context-aware embeddings by considering the entire sentence bidirectionally (i.e., both left-to-right and right-to-left).
- Unlike traditional word embeddings like Word2Vec or GloVe, which generate a single representation for each word, BERT produces different embeddings for each word based on its context.
- **Pros:** Pre-training and Fine-tuning, Contextualized Embeddings, Bi-directional Context.
- **Cons:** Complexity and Size, Inference Time.

What is a Large Language Model (LLM)

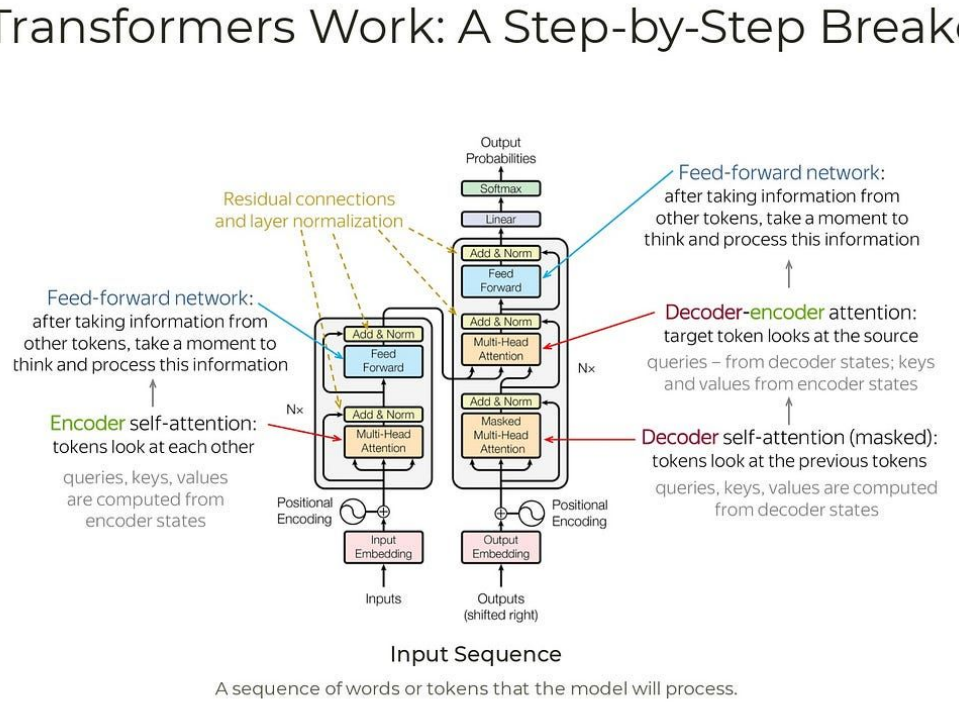
Large Language Models (LLMs) are advanced AI systems built on deep neural networks designed to process, understand and generate human-like text.

- LLMs learn patterns, grammar and context from text and can answer questions, write content, translate languages and many more.
- By using massive datasets and billions of parameters, LLMs have transformed the way humans interact with technology.
- Modern LLMs include ChatGPT (OpenAI), Google Gemini, Anthropic Claude, etc.

What is a Large Language Model (LLM)



Architecture of Large Language Model (LLM)



Working of LLM

A standard Transformer has two main parts:

Encoder

- Reads and understands the input text
- Converts words into meaningful representations
- An **encoder** converts input text into an intermediate representation. An encoder is an enormous **neural net**.

Decoder

- Generates output text (used in tasks like translation or text generation)
- A **decoder** converts that intermediate representation into useful text. A decoder is also an enormous neural net.
- **In LLMs:** Models like GPT use **only the decoder,** Models like BERT use **only the encoder**

Working of LLM

Every text-generative Transformer consists of these **three key components:**

1. **Embedding**: Text input is divided into **smaller units called tokens**, which can be words or subwords. These **tokens are converted into numerical vectors** called embeddings, which capture the semantic meaning of words.
2. **Transformer Block** is the **fundamental building block of the model** that processes and transforms the input data. Each block includes:
    - **Attention Mechanism**, the core component of the Transformer block. It allows tokens to communicate with other tokens, capturing contextual information and relationships between words.

Working of LLM (Cont'd..)

- **MLP (Multilayer Perceptron) Layer**, a feed-forward network that operates on each token independently. While the goal of the attention layer is to route information between tokens, the goal of the MLP is to refine each token's representation.

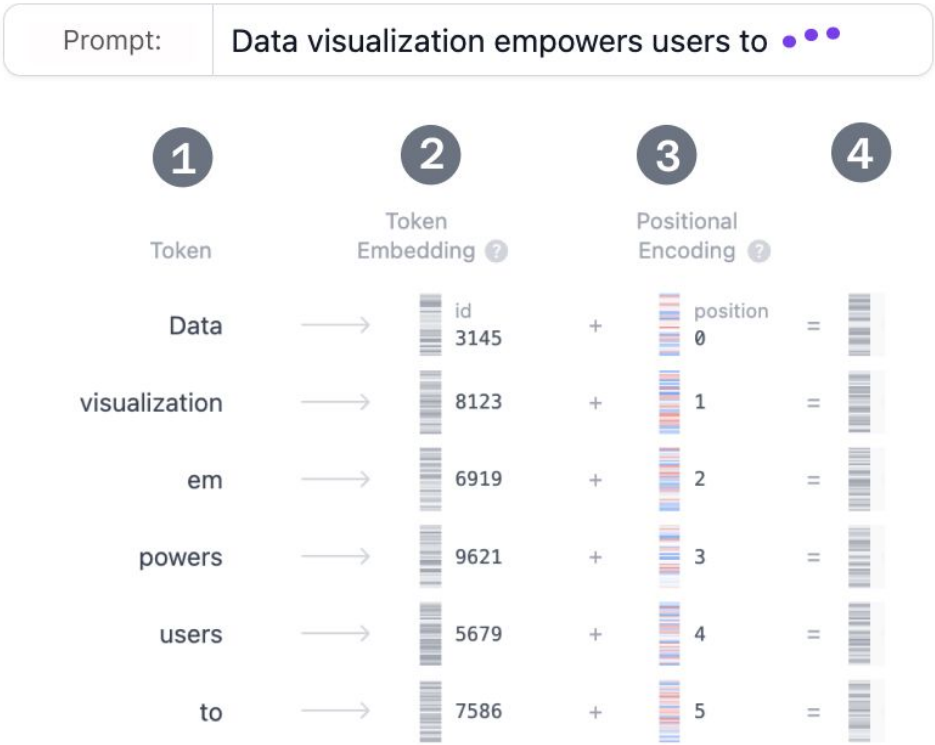
3. Output Probabilities: The final linear and softmax layers transform the processed embeddings into probabilities, enabling the model to make predictions about the next token in a sequence.

Working of LLM (Cont'd..)

- **Input Prompt: "Data visualization empowers users to"**
- It transforms the text into a numerical representation that the model can work with. To convert a prompt into embedding, we need to
  - 1) tokenize the input,
  - 2) obtain token embeddings,
  - 3) add positional information, and finally
  - 4) add up token and position encodings to get the final embedding

Eg.: GPT-2's vocabulary has 50,257 unique tokens. GPT-2 (small) represents each token in the vocabulary as a 768-dimensional vector; the dimension of the vector depends on the model. These embedding vectors are stored in a matrix of shape (50,257, 768), containing approximately 39 million parameters!

Working of LLM (Cont'd..)



Working of LLM (Cont'd..) - Transformer Block

Multi-Head Self-Attention

- The self-attention mechanism enables the model to capture relationships among tokens in a sequence, so that each token's representation is influenced by the others.
- Multiple attention heads allow the model to consider these relationships from different perspectives;
- for example, one head may capture short-range syntactic links while another tracks broader semantic context.

Working of LLM (Cont'd..) - Transformer Block

Each token's embedding vector is transformed into three vectors: Query (Q), Key (K), and Value (V). These vectors are derived by multiplying the input embedding matrix with learned weight matrices for Q, K, and V. Here's a web search analogy to help us build some intuition behind these matrices:

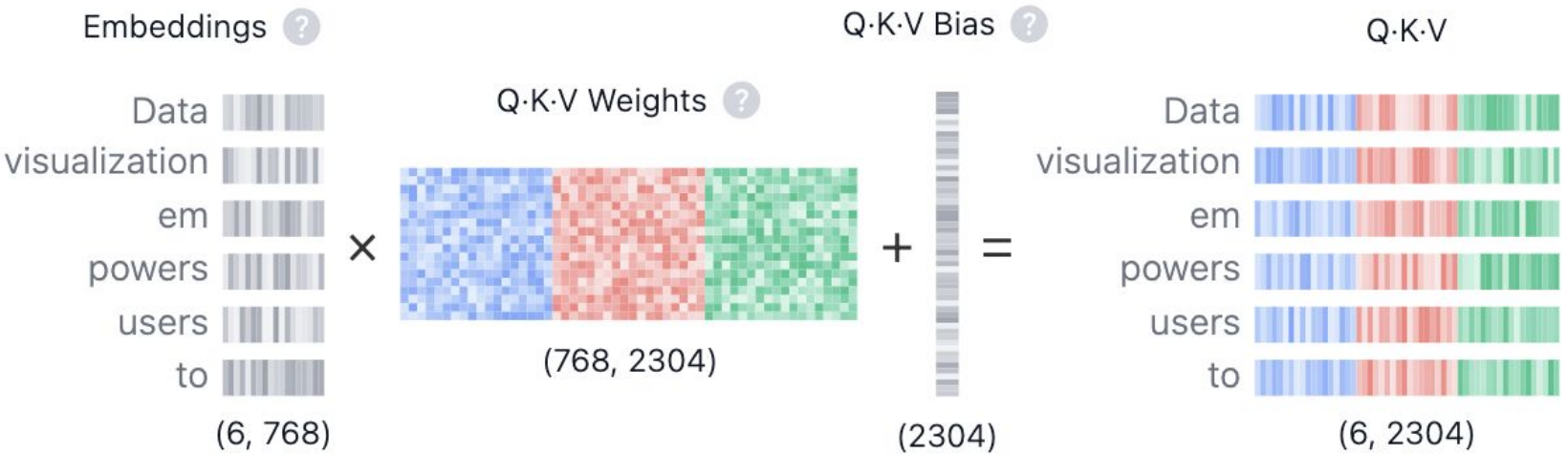
- Query (Q) is the search text you type in the search engine bar. This is the token you want to *"find more information about"*.
- Key (K) is the title of each web page in the search result window. It represents the possible tokens the query can attend to.

Working of LLM (Cont'd..) - Transformer Block

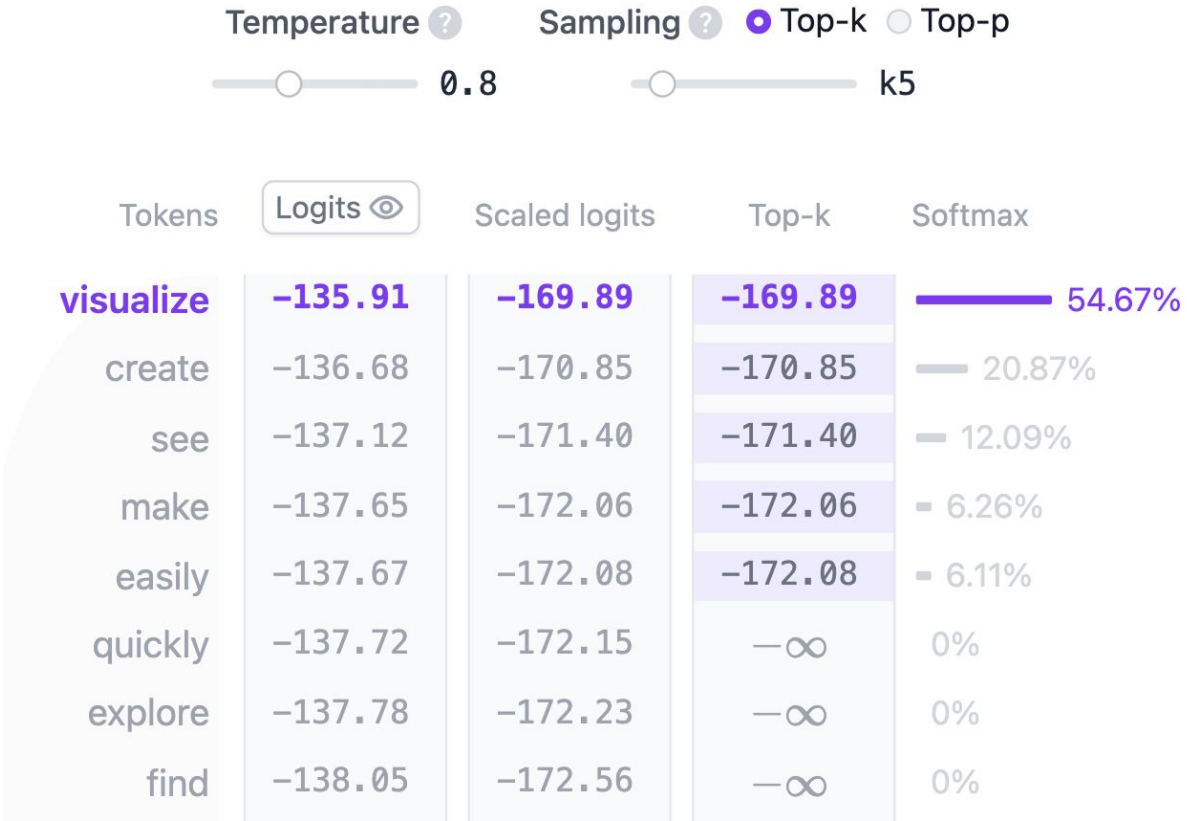
- Value (V) is the actual content of web pages shown. Once we matched the appropriate search term (Query) with the relevant results (Key), we want to get the content (Value) of the most relevant pages.

By using these QKV values, the model can calculate attention scores, which determine how much focus each token should receive when generating predictions.

Working of LLM (Cont'd..) - Transformer Block



Working of LLM (Cont'd..) - Output Prob



Popular LLMs

Some of the most widely used LLMs include:

- **GPT-4 and GPT-4o (OpenAI)**: Advanced multimodal reasoning and dialogue capabilities.
- **Gemini 1.5 (Google DeepMind)**: Long-context reasoning, capable of handling 1M+ tokens.
- **Claude 3 (Anthropic)**: Safety-focused, strong at reasoning and summarization.
- **LLaMA 3 (Meta)**: Open-weight model, popular in research and startups.
- **Mistral 7B / Mixtral (Mistral AI)**: Efficient open-source alternatives for developers.
- **BERT and RoBERTa (Google/Facebook)**: Strong embedding models for NLP tasks.
- **mBERT and XLM-R**: Early multilingual LLMs.
- **BLOOM**: Large open-source multilingual model, collaboratively developed.

Attention Mechanism in ML

- It allows models to focus on the most important parts of input data when making predictions.
- It assigns different weights to different elements hence helping the model prioritize relevant information instead of treating all inputs equally.
- It forms the foundation of advanced models like Transformers and BERT and is widely used in Natural Language Processing (NLP) and Computer Vision.

Working: Attention Mechanism

Step 1: Input Encoding: The input sequence is first encoded using an encoder like RNN, LSTM, GRU or Transformer to generate hidden states representing the input context.

Step 2: Query, Key and Value Vectors: Each input is transformed into:

- **Query (Q):** Represents what we're looking for.
- **Key (K):** Represents what information each input contains.
- **Value (V):** Contains the actual information of each input.

These are linear transformations of the input embeddings.

Working: Attention Mechanism

Step 3: Key–Value Pair Creation: Each input is represented as a pair:

- **Key (K):** Represents the "address" or identifier of information.
- **Value (V):** Represents the actual content.

Step 4: Similarity Computation: The model computes similarity between the query and each key to determine relevance.

$$\text{Score}(s,i) = \begin{cases} h_s \cdot y_i & \text{(Dot Product)} \\ h_s^T W y_i & \text{(General)} \\ v^T \tanh(W[h_s; y_i]) & \text{(Concat)} \end{cases}$$

Where:

- h_s: Encoder hidden state at position s
- y_i: Decoder hidden state at position i
- W: Weight matrix
- v: Weight vector

Working: Attention Mechanism

Step 5: Attention Weights Calculation: The similarity scores are passed through a softmax function to convert them into attention weights:

$$\alpha(s,i) = \text{softmax}(\text{Score}(s,i))$$

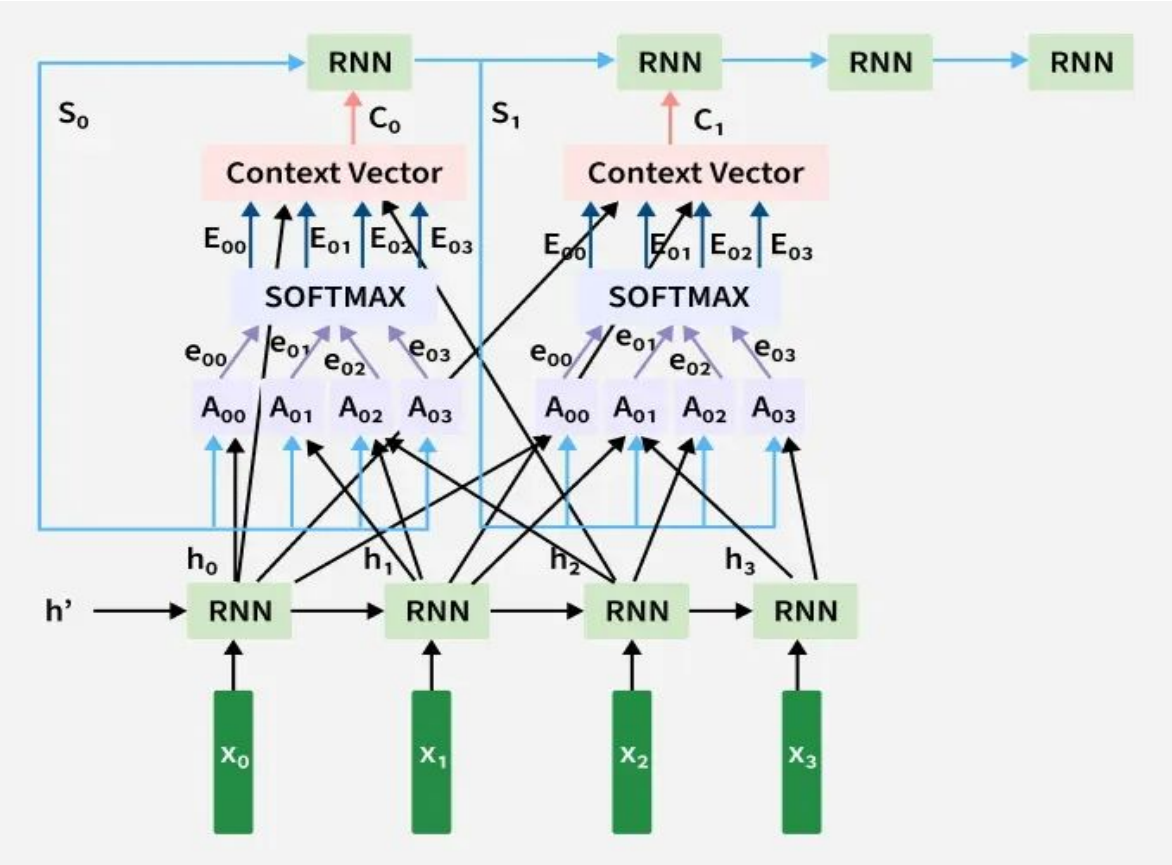
Step 6: Weighted Sum: The attention weights are used to compute a weighted sum of the value vectors:

$$c_t = \sum_{i=1}^{T_s} \alpha(s,i) h_i$$

Step 7: Context Vector: The context vector c_t summarizes the most relevant information from the input sequence and is fed to the decoder.

Step 8: Integration: The decoder uses both its own hidden state and the context vector to generate the next output token

Attention Mechanism Architecture



How Attention Mechanism Improves Traditional Deep Learning Models

Traditional deep learning models like RNNs, LSTMs and CNNs have limitations when handling long or complex dependencies. The attention mechanism enhances their effectiveness as follows:

- **RNNs/LSTMs:** These models compress the entire input into one vector, causing information loss over long sequences. Attention dynamically focuses on the relevant parts, resolving long-term dependency issues.
- **CNNs:** CNNs have fixed receptive fields and attention enables global dependencies, helping capture relationships beyond local patterns.
- **Seq2Seq Models:** Replace single context vectors with multiple dynamic ones, improving translation accuracy.
- **General Advantage:** Helps assign different importance weights to inputs, avoiding equal treatment of all tokens or features.

Applications

- **Machine Translation:** Focuses on relevant words while generating each output word.
- **Text Summarization:** Selects key information for concise summaries.
- **Image Captioning:** Attends to specific image regions to describe them accurately.
- **Sentiment Analysis & NER**: Highlights important words or entities in text.
- **Speech Recognition**: Focuses on critical audio frames for better transcription.

Advantages

- Helps models focus dynamically on the most relevant information.
- Solves long-term dependency issues in sequential data.
- Improves performance and interpretability in NLP and Vision tasks.
- Enables parallel computation (in self-attention) unlike RNNs.
- Enhances context understanding in transformer-based models.

Limitations

- Computationally expensive for long sequences (especially self-attention).
- Requires large memory due to quadratic complexity.
- Attention weights can be difficult to interpret in large models.
- Needs large datasets for effective training.

Dialogue Systems and Chatbots

- Personal Assistants on phones or other devices
  - SIRI, Alexa, Cortana, Google Assistant
- Playing music, setting timers and clocks
- Chatting for fun
- Booking travel reservations
- Clinical uses for mental health

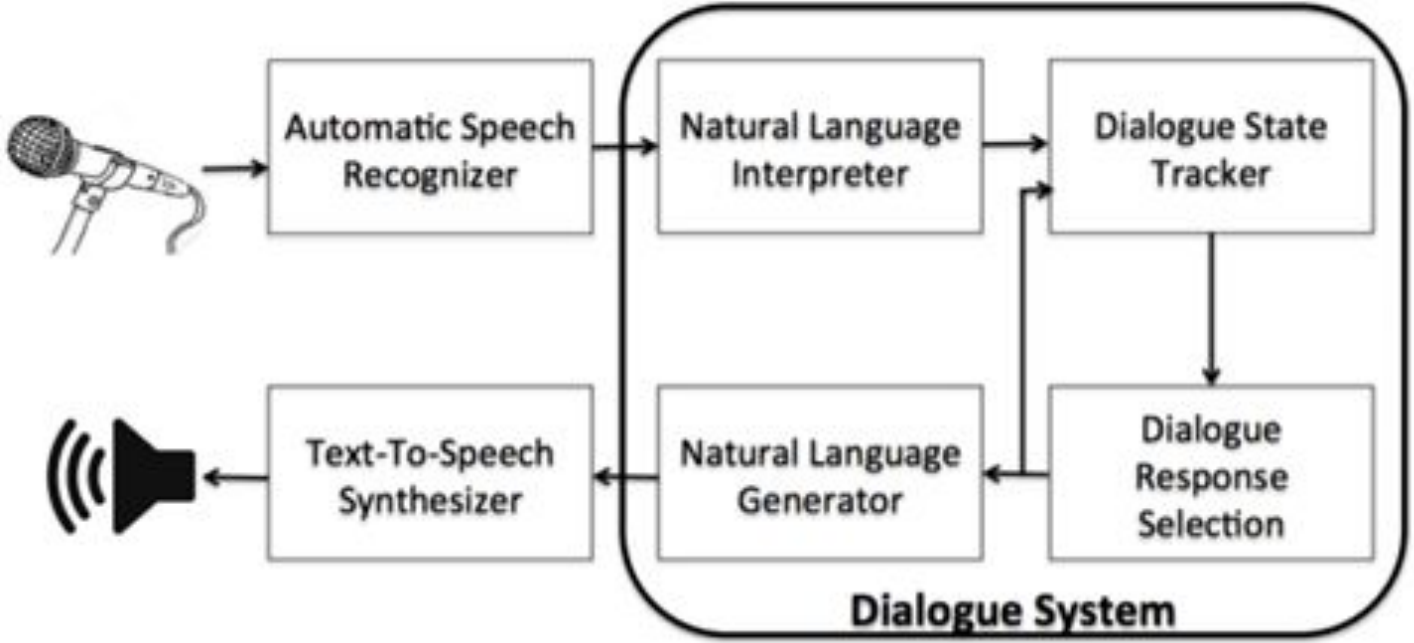
Dialogue Systems and Chatbots

- Two kind of conversational agents

  1. Chatbots- mimic informal human chatting- for fun, or even for therapy

  2. (Task-based) Dialogue Agents- interfaces to personal assistants- cars, robots, appliances- booking flights or restaurants

Dialogue Systems and Chatbots



Information Retrieval and RAG

- User has an information need
- And has some collection of documents
- User wants to find a relevant document
  - a document (or documents)
  - in the collection
  - that satisfy their need

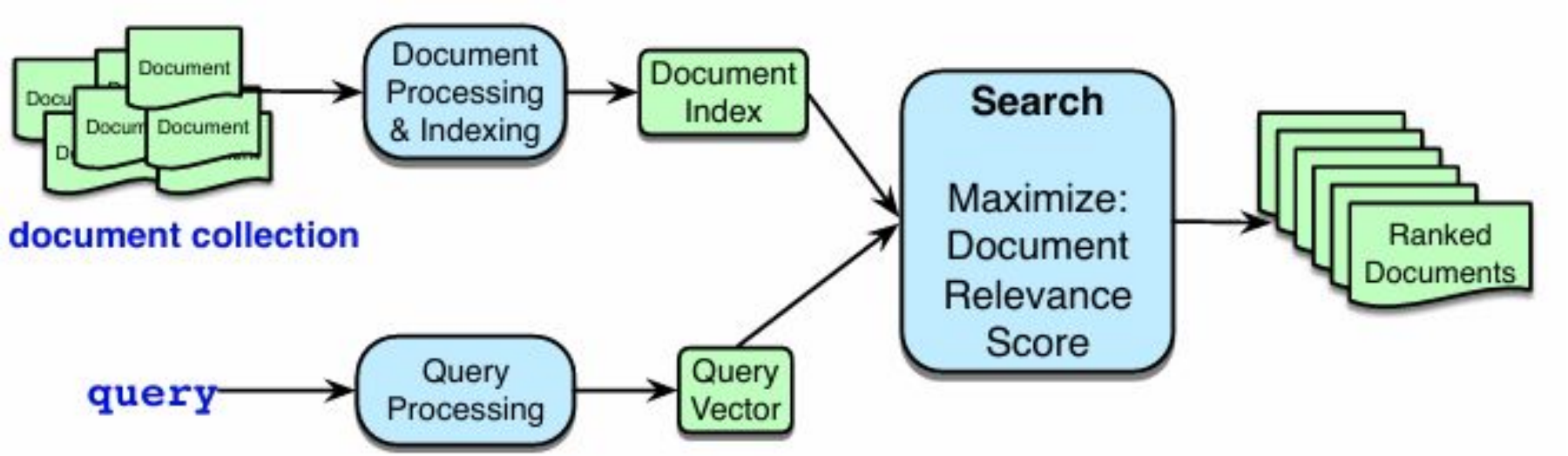
The vector space model of IR

1. Represent each document as a vector of counts of the words it contains.
2. Represent a query as a vector too
3. Return the document whose vector is most similar to the query vector

Ranked Retrieval

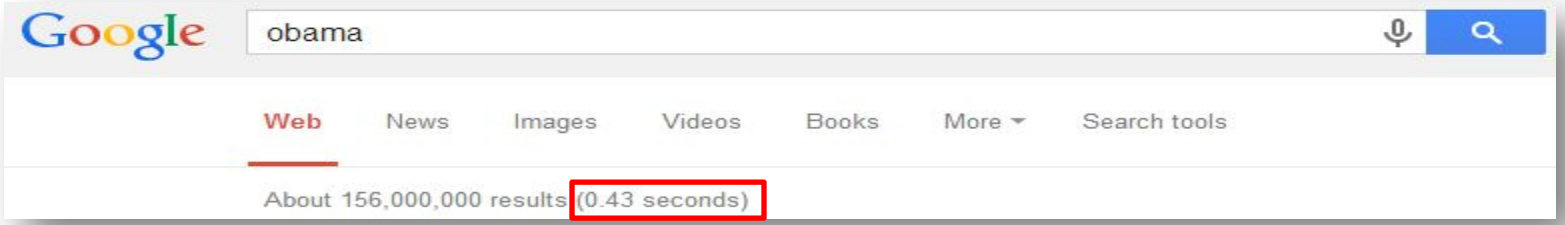
- The retriever returns top-k documents
- These are ranked
- We can show the user these, or some subset

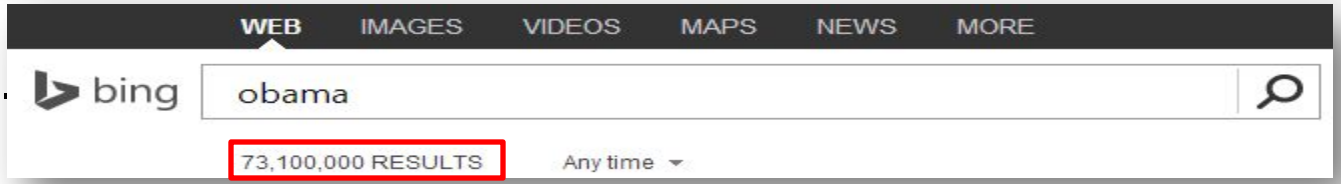
Retrieval


Document
Document
Document
document collection
Document Processing & Indexing
Document Index
Search
Maximize: Document Relevance Score
Ranked Documents
query
Query Processing
Query Vector

Which search engine do you prefer: Bing or Google?

- What are your judging criteria?
  - How fast does it respond to your query?





Collection and Documents

The curse of the black pearl
Ship Jack Sparrow Caribbean Turner Elizabeth Gun Fight

Finding Nemo
Ocean Fish Nemo Reef Animation

Tintin
Ocean Animation Ship Haddock Tintin

Titanic
Ship Rose Jack Atlantic Ocean England Sink

The Dark Knight
Bruce Wayne Batman Joker Harvey Gordon Gun Fight Crime

Skyfall
007 James Bond MI6 Gun Fight

Silence of the Lambs
Hannibal Lector FBI Crime Gun Cannibal

The Ghost Ship
Ship Ghost Ocean Death Horror

- Document: unit of retrieval
- Collection: the group of documents from which we retrieve
  - Also called *corpus* (a body of texts)

Boolean retrieval

- Find all documents containing a word w
- Find all documents containing a word w_1 but not containing the word w_2
- Queries in the form of any Boolean expression
- Query: **Jack**

Boolean retrieval

- Find all documents containing a word w
- Find all documents containing a word w_1 but not containing the word w_2
- More complicated Boolean queries
- Query: **Jack**

Term – document matrix

	Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship
Ship	1	0	1	1	0	0	0	1
Jack	1	0	0	1	0	0	0	0
Bond	0	0	0	0	0	1	0	0
Gun	1	0	0	0	1	1	1	0
Ocean	1	1	1	1	0	0	0	1
Captain	1	0	1	1	0	0	0	0
Batman	0	0	0	0	1	0	0	0
Crime	0	0	0	0	1	0	1	0

- The entry $(w, d) = 1$ if and only if the word w is present in document d
- Terms are dimensions of this matrix (*units of index; we will discuss later*)
- Commonly called *term – document matrix*
- Term and word are not same, though often words are used as terms

Boolean retrieval

	Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship
Ship	1	0	1	1	0	0	0	1
Jack	1	0	0	1	0	0	0	0
Bond	0	0	0	0	0	1	0	0
Gun	1	0	0	0	1	1	1	0
Ocean	1	1	1	1	0	0	0	1
Captain	1	0	1	1	0	0	0	0
Batman	0	0	0	0	1	0	0	0
Crime	0	0	0	0	1	0	1	0

- Query: Jack
- Results: 10010000

Boolean retrieval

	Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship
Ship	1	0	1	1	0	0	0	1
Jack	1	0	0	1	0	0	0	0
Bond	0	0	0	0	0	1	0	0
Gun	1	0	0	0	1	1	1	0
Ocean	1	1	1	1	0	0	0	1
Captain	1	0	1	1	0	0	0	0
Batman	0	0	0	0	1	0	0	0
Crime	0	0	0	0	1	0	1	0

- Query: Captain AND Gun
- Results: 10110000 && 10001110 = 10000000

Query and relevant documents

- Query: given by user, represents the *information need*
  - Information need is the topic, conceptually what the user wants to know
  - Query is the representation of information need that the user conveys to the retrieval system
- Relevant document: a document that satisfies the information need, as perceived by the user
  - Merely matching the query terms does not mean a document is relevant
  - A relevant document must satisfy the actual information need

Precision and recall

- Precision: what fraction of the returned results are relevant?
  - Given a query q and a document d, need a judgment whether d is relevant for q

- Recall: what fraction of the relevant documents in the collection were returned by the system?
  - Given a query q, need the set D_q of all relevant documents that are relevant to q

What if the collection is “large”?

	Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship
Ship	1	0	1	1	0	0	0	1
Jack	1	0	0	1	0	0	0	0
Bond	0	0	0	0	0	1	0	0
Gun	1	0	0	0	1	1	1	0
Ocean	1	1	1	1	0	0	0	1
Captain	1	0	1	1	0	0	0	0
Batman	0	0	0	0	1	0	0	0
Crime	0	0	0	0	1	0	1	0

- About 1 million documents (still not so large)
- About 500,000 distinct terms
- A term – document matrix of 500,000 × 1 million Boolean entries ~ 500GB

What if the collection is “large”?

	Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship
Ship	1	0	1	1	0	0	0	1
Jack	1	0	0	1	0	0	0	0
Bond	0	0	0	0	0	1	0	0
Gun	1	0	0	0	1	1	1	0
Ocean	1	1	1	1	0	0	0	1
Captain	1	0	1	1	0	0	0	0
Batman	0	0	0	0	1	0	0	0
Crime	0	0	0	0	1	0	1	0

Sparse matrix → inverted index

	Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship
Ship	1		1	1				1
Jack	1			1				
Bond						1		
Gun	1				1	1	1	
Ocean	1	1	1	1				1
Captain	1		1	1				
Batman					1			
Crime					1		1	

- In reality term – document matrices are very sparse
- Most terms are NOT present in most documents
- For every term, store only the documents where the term is present

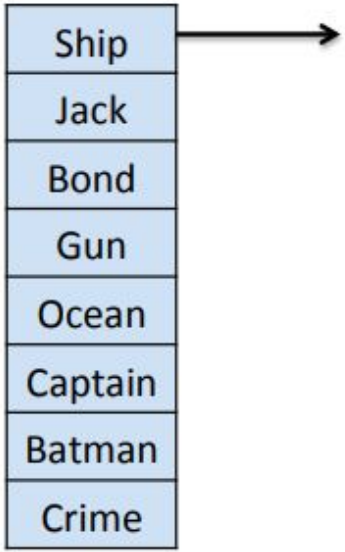
Sparse matrix → inverted index

	1 Black pearl	2 Finding Nemo	3 Tintin	4 Titanic	5 Dark Knight	6 Skyfall	7 Silence of lambs	8 Ghost ship
Ship	1		1	1				1
Jack	1			1				
Bond						1		
Gun	1				1	1	1	
Ocean	1	1	1	1				1
Captain	1		1	1				
Batman					1			
Crime					1		1	

- Represent documents by document IDs

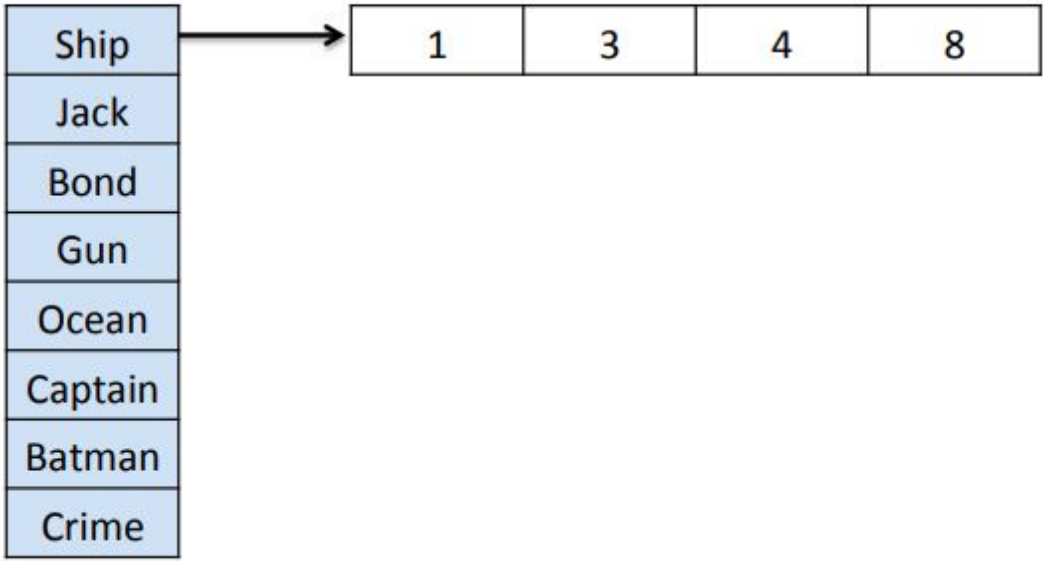
Sparse matrix → inverted index

1	2	3	4	5	6	7	8
Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship



Sparse matrix → inverted index

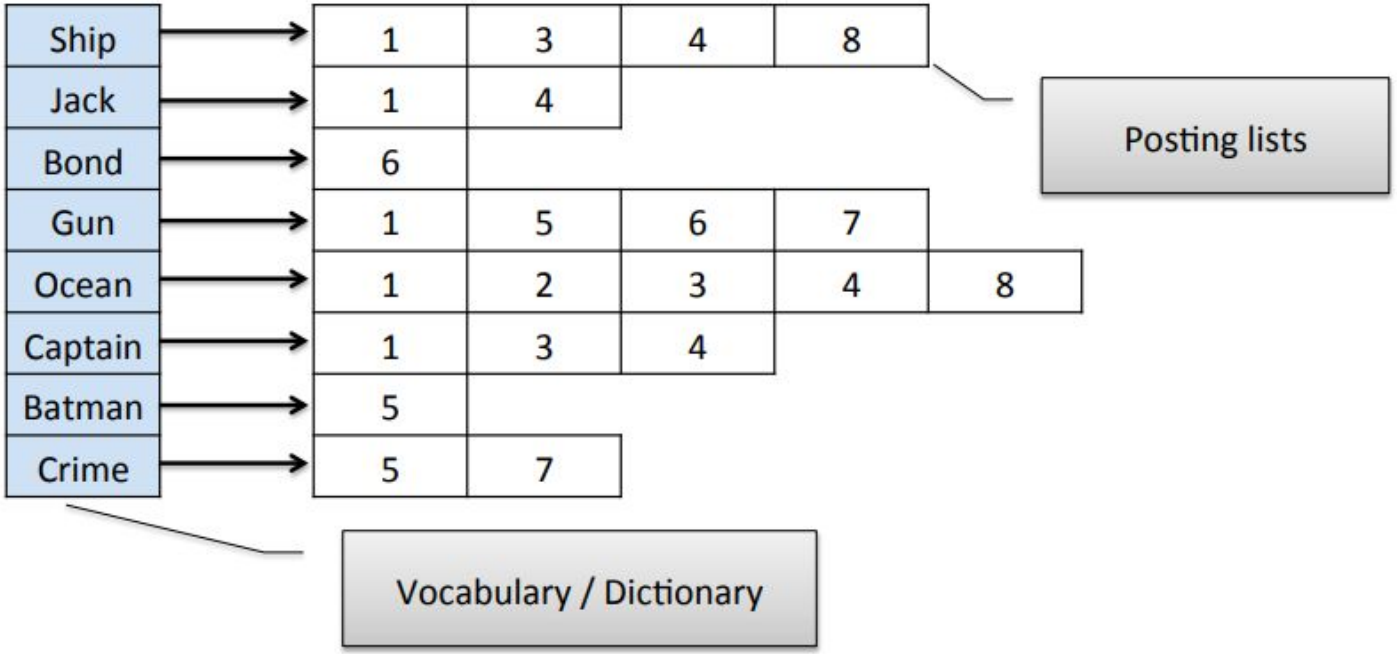
1	2	3	4	5	6	7	8
Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship



Sparse matrix → inverted index

1	2	3	4	5	6	7	8
Black pearl	Finding Nemo	Tintin	Titanic	Dark Knight	Skyfall	Silence of lambs	Ghost ship

Inverted Index



Creating an inverted index

Term	docId
Ship	1
Captain	1
Jack	1
…	…
Ship	3
Tintin	3
…	…
Jack	4
…	…

Sort by term

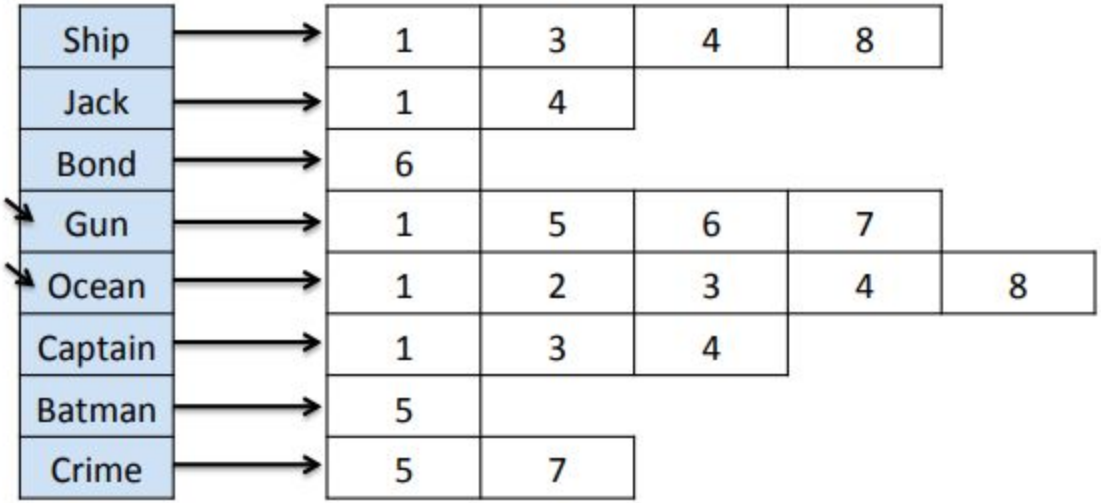
Term	docId
…	…
Captain	1
…	…
Jack	1
Jack	4
…	…
Ship	1
Ship	3
…	…

- For each document, write out pairs (term, docid)
- Sort by term, then group

Term	docId	docId	docId
Captain	1	…	
Jack	1	4	…
Ship	1	3	…
…	…		

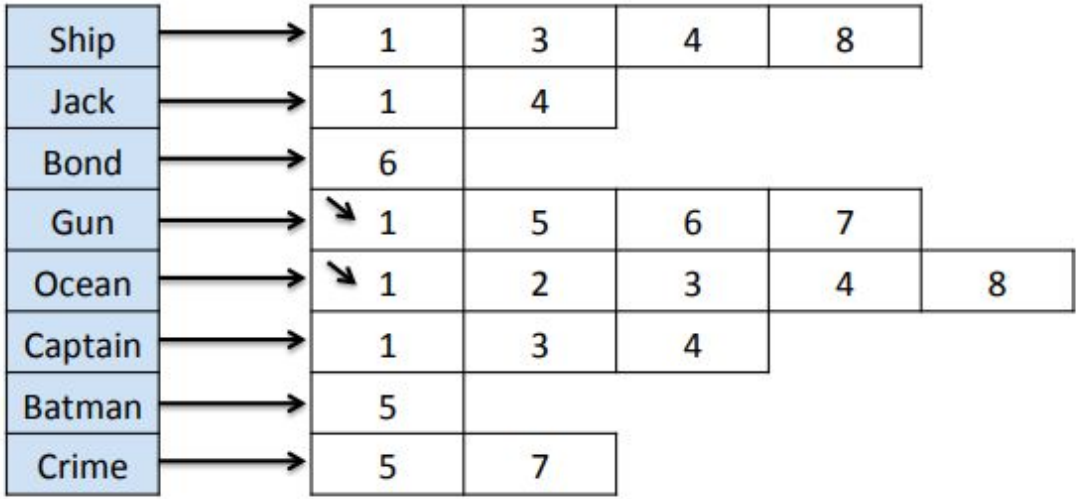
Boolean query processing

Query: Gun OR Ocean



Boolean query processing

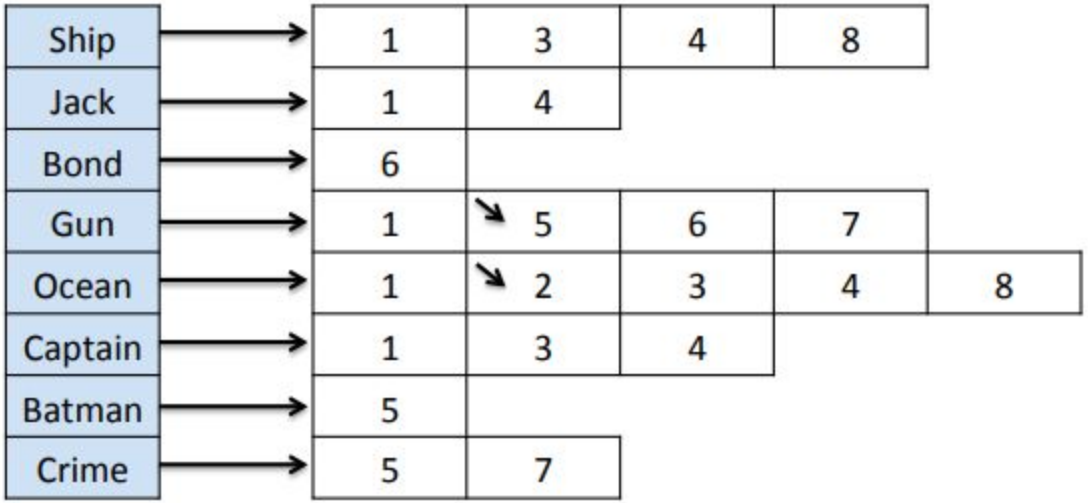
Query: Gun OR Ocean



Results: 1

Boolean query processing

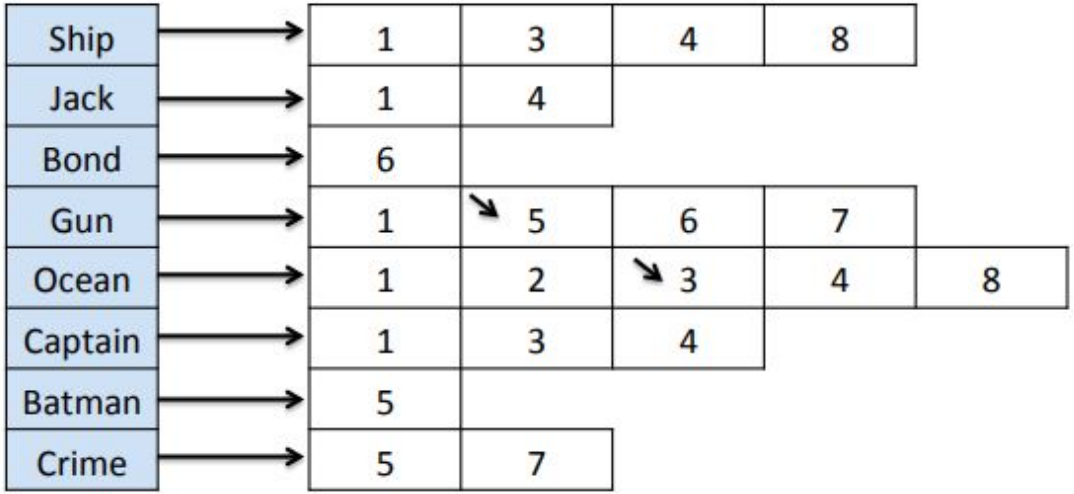
Query: Gun OR Ocean



Results: 1 2

Boolean query processing

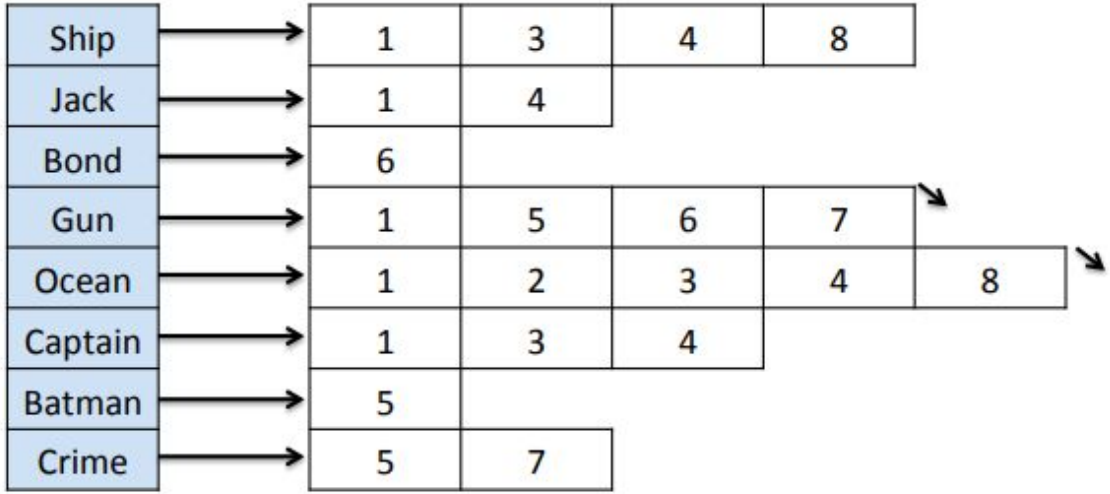
Query: Gun OR Ocean



Results: 1 2 3

Boolean query processing

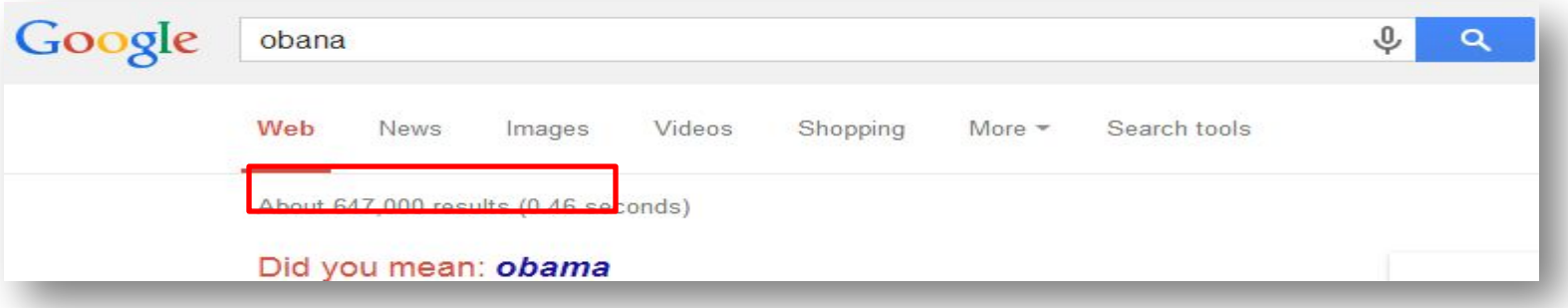
Query: Gun OR Ocean



Results: 1 2 3 4 5 6 7 8

Which search engine do you prefer: Bing or Google?

- What are your judging criteria?
  - Can it correct my spelling errors?



  - Can it suggest me good related queries?

Not just the web

Searching our email

Searching corporate documents

Searching personal medical records

Retrieval evaluation

- Evaluation criteria are all good, but not essential
  - Goal of any IR system
    - Satisfying users' information need
  - Core quality measure
    - *"how well a system meets the information needs of its users." – wiki*
    - Unfortunately vague and hard to execute

Quantify the IR quality measure

- Information need
  - *“an individual or group's desire to locate and obtain information to satisfy a conscious or unconscious need” – wiki*
  - Reflected by user query

Quantify the IR quality measure

- Satisfaction
  - *“the opinion of the user about a specific computer application, which they use” – wiki*
  - Reflected by
    - Increased result clicks
    - Repeated/increased visits
    - Result relevance

Classical IR evaluation

- Cranfield experiments
  - Pioneer work and foundation in IR evaluation
  - Basic hypothesis
    - Retrieved documents' relevance is a good proxy of a system's utility in satisfying users' information need
  - Procedure
    - 1,398 abstracts of aerodynamics journal articles
    - 225 queries
    - Exhaustive relevance judgments of all (query, document) pairs

Classical IR evaluation

- Three key elements for IR evaluation
  1. A document collection
  2. A test suite of information needs, expressible as queries
  3. A set of relevance judgments, e.g., binary assessment of either *relevant* or *non-relevant* for each query-document pair

Evaluation metric

- To answer the questions
  - Is Google better than Bing?
  - Which ranking method is the most effective?
  - Shall we perform stemming or stopword removal?
- We need a quantifiable metric, by which we can compare different IR systems
  - As unranked retrieval sets
  - As ranked retrieval results

Evaluation of unranked retrieval sets

- In a Boolean retrieval system
  - Precision: fraction of retrieved documents that are relevant, i.e., p(relevant|retrieved)
  - Recall: fraction of relevant documents that are retrieved, i.e., p(retrieved|relevant)

	relevant	nonrelevant
retrieved	true positive (TP)	false positive (FP)
not retrieved	false negative (FN)	true negative (TN)

Precision: $P = \frac{TP}{TP + FP}$

Recall: $R = \frac{TP}{TP + FN}$

Evaluation of unranked retrieval sets

- Precision and recall trade off against each other
  - Precision decreases as the number of retrieved documents increases (unless in perfect ranking), while recall keeps increasing
  - These two metrics emphasize different perspectives of an IR system
    - Precision: prefers systems retrieving fewer documents, but highly relevant

Evaluation of ranked retrieval results

- Summarize the ranking performance with a single number
  - Binary relevance
    - Eleven-point interpolated average precision
    - Precision@K (P@K)
    - Mean Average Precision (MAP)
    - Mean Reciprocal Rank (MRR)
  - Multiple grades of relevance
    - Normalized Discounted Cumulative Gain (NDCG)

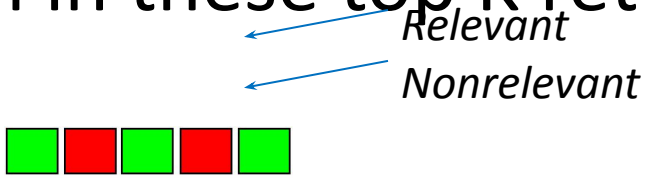
Precision@K

- Set a ranking position threshold K
- Ignores all documents ranked lower than K
- Compute precision in these top K retrieved documents
  - E.g.,

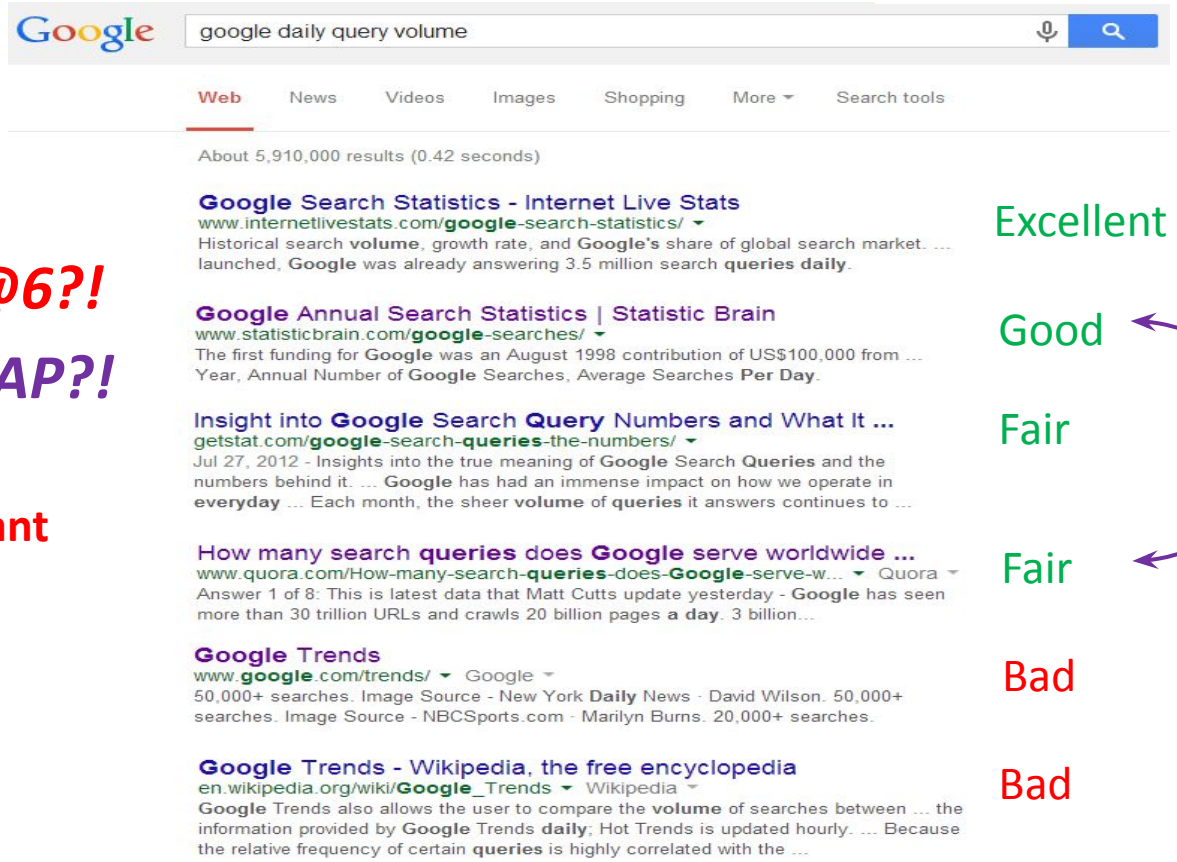
    P@3 of 2/3

    P@4 of 2/4

    P@5 of 3/5



Beyond binary relevance



P@6
MAP
MRR

Same P@6?!

Same MAP?!

Relevant
Nonrelevant

Ranked retrieval

The retriever returns top-k documents

These are ranked

We can show the user these, or some subset.

Two architectures for measuring query-doc relevance (= similarity)

Sparse retrieval

- represent query and doc as **vectors of word counts**
- weighted by tf-idf, BM25

Dense retrieval

- Use LLM to represent query and doc as **embeddings**

In both cases, similarity is usually the **cosine** between query and document representations

Information Retrieval in the LLM Age

If we have some information needs we don't need a document collection!

- Where is the Louvre Museum located?
- Where does the energy in a nuclear explosion come from?
- How to get a script l in latex?

Just prompt an LLM!

Information Retrieval in the LLM Age

But IR is still relevant for LLMs!

Retrieving relevant documents can still help solve problems that LLMs have with meeting information needs!

What are these problems?

LLMs Hallucinate

Hallucination: a response that is not faithful to the facts of the world.

In the legal domain LLMs were shown to hallucinate up to 88% of the time!

Dahl et al. (2024)

Can't use Proprietary Data

People need to ask questions about:

- personal email
- healthcare applications to medical records
- internal corporate documents
- legal documents discovery

These are (hopefully) not in the pretraining data of large language models!

Can't Handle Dynamic Data

LLMs can't answer questions about rapidly changing information like

- the sports match that happened last week
- the earthquake that happened this morning
- the latest Oscar winner

In general, data shifts over time!

Solution: RAG

Retrieval-Augmented Generation

1. Use IR to **retrieve** documents from some collection
2. Then use LLM to **generate** an answer conditioned on the documents

Problem with classic IR

The **vocabulary mismatch problem**

- tf-idf cosine similarities measure query-doc similarity only if there is **exact word overlap** between query and doc!
- But query-writer can't know the exact words the doc might include!
  - Maybe query says "premiere" and document says "opening night"?

Dense retrieval

We still compare document vectors with query vectors to rank documents

But instead of representing query and documents with count vectors

- We represent both with dense **embeddings**!
- Here's a quick preview of embeddings; more details in future lectures

IR plays a central role in modern LLMs

LLMs can answer some information needs without a document collection!

- Where is the Louvre Museum located?
- Where does the energy in a nuclear explosion come from?
- How to get a script l in latex?

Just prompt an LLM!

But there are issues we mentioned earlier!

LLMs hallucinate, giving answers that aren't faithful to the facts of the world.

- They can make up medical, legal facts

LLMs can't use proprietary data

- Email, medical records, corporate docs

LLMs can't handle dynamic data

- Recent news, earthquakes, sports events

Solution: RAG

Retrieval-Augmented Generation

1. Use IR to **retrieve** documents from some collection
2. Then use LLM to **generate** an answer conditioned on the documents

Basic RAG

Given a document collection D and a user query q

- Call a retriever to return top *k* passages
- Create a prompt that includes *q* and the passages
- Call an LLM with the prompt

Extensions: Agent-based RAG

Instead of running RAG automatically on every user turn

Have a **retrieval agent**

System decides when to call the retrieval agent and for which document collection

Extensions: Training

Instruction-tune an LLM on a dataset of questions with retrieved passages and correct answers

Test-time compute: prompt an LLM to answer the question and simultaneously to generate reflections on which passages were useful

Two kinds of question answering datasets

Natural information-seeking questions

- Someone actually wanted to know the answer to this

Probing (testing) questions

- Exam-type questions for LLM evaluation

Natural Questions (Kwiatkowski et al., 2019),

anonymized English **queries** to the Google search engine and short and long **answers** (hand-created from Wikipedia)

"When are hops added to the brewing process?"

short answer: *the boiling process*

long answer: paragraph from the Wikipedia page on *Brewing*

MS MARCO (Microsoft Machine Reading Comprehension) collection of datasets,

1 million real anonymized English questions from Microsoft Bing query logs

human generated answer

9 million passages (Bajaj et al., 2016)